UNIVERSITY OF DERBY

5CM506
Data Driven Systems

Database Programming
Transact-SQL (T-SQL)

Dr. Maqbool Hussain

UNIVERSITY OF DERBY

Aim

1. Understand key programming artifacts

1. Using View to manage and secure the database

1. Optimise business logic – introducing programming at database end.

UNIVERSITY OF DERBY

Overview - key programmable artifacts

T-SQL:

- T-SQL (Transact-SQL) is a set of programming extensions from Sybase and Microsoft that add several features to the SQL.
- It covers transaction control, exception and error handling, row processing and declared variables.

T-SQL and SQL:

- T-SQL is extension of SQL.
- T-SQL support procedural programming and local variable, while SQL does not.
- T-SQL is proprietary, while SQL is an open format.

UNIVERSITY OF DERBY

Variables in T-SQL

Declaring variable:

- Syntax: `DECLARE @varname datatype;`
- Example: `DECLARE @name nvarchar(30);`

Value assignment to variable:

- Syntax: `SET @varname = value;`
- Example: `SET @name = 'MAQBOOL'`

UNIVERSITY OF DERBY

Variables in T-SQL – Example Adding two values and printing outcomes

```
Use Test;

-- Declaring and printing variable value
DECLARE @num1 int;
DECLARE @num2 int;
DECLARE @sum int, @sumOut nvarchar(40);

SET @num1 = 5;
SET @num2 = 3;

SET @sum = @num1 + @num2;
SET @sumOut = CAST(@num1 as nvarchar(10)) +
  N'+' + CAST(@num2 as nvarchar(10)) + N'=' +
  CAST(@sum as nvarchar(10));

print @sumOut;
```

OUTPUT: 5+3=8

UNIVERSITY OF DERBY

Transaction and T-SQL

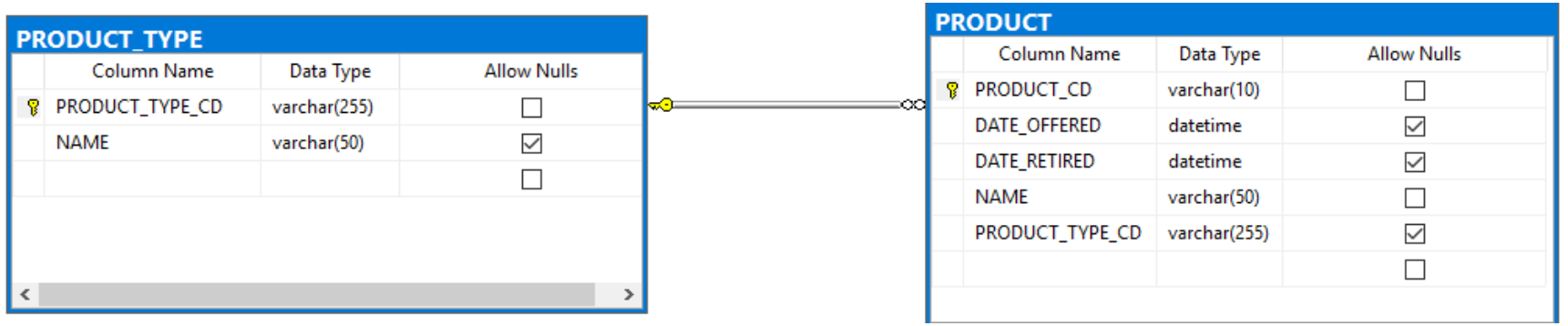
- Transaction encapsulate logical related set of operations – which ensures either all operations are executed successful or rollback all if any failed.
- For example: Sending £100 from Account A to Account B.
  - Operation 1: A = A -100;
  - Operation 2: B = B + 100

If Operation 2 failed then – operation 1 will be rollbacked with A = A + 100.

Transaction Example

Consider Products schema and with existing data:

- Adding new product type ‘DISCOUNT’ and product ‘Package Loan’



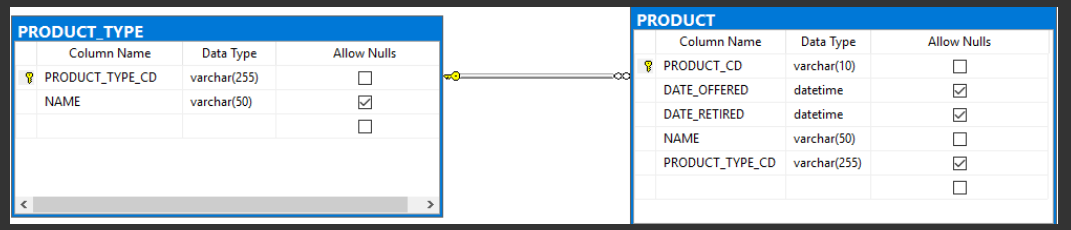

```
SELECT * FROM PRODUCT_TYPE
SELECT * FROM PRODUCT
```

	PRODUCT_TYPE_CD	NAME
1	ACCOUNT	Customer Accounts
2	INSURANCE	Insurance Offerings
3	LOAN	Individual and Business Loans

	PRODUCT_CD	DATE_OFFERED	DATE_RETIRED	NAME	PRODUCT_TYPE_CD
1	AUT	2000-01-01 00:00:00.000	NULL	auto loan	LOAN
2	BUS	2000-01-01 00:00:00.000	NULL	business line of credit	LOAN
3	CD	2000-01-01 00:00:00.000	NULL	certificate of deposit	ACCOUNT
4	CHK	2000-01-01 00:00:00.000	NULL	checking account	ACCOUNT
5	MM	2000-01-01 00:00:00.000	NULL	money market account	ACCOUNT
6	MRT	2000-01-01 00:00:00.000	NULL	home mortgage	LOAN
7	SAV	2000-01-01 00:00:00.000	NULL	savings account	ACCOUNT
8	SBL	2000-01-01 00:00:00.000	NULL	small business loan	LOAN

Transaction Example : Failed or Pass

```
USE Test;
-- Begin the transaction
BEGIN TRANSACTION

INSERT INTO [dbo].[PRODUCT_TYPE]
  ([PRODUCT_TYPE_CD]
  ,[NAME])
VALUES
  ('DISCOUNT','Special Offers')

-- Check the insert is executed without any error.
IF @@ERROR <> 0
  GOTO FailedTran;


INSERT INTO [dbo].[PRODUCT]
  ([PRODUCT_CD]
  ,[DATE_OFFERED]
  ,[DATE_RETIRED]
  ,[NAME]
  ,[PRODUCT_TYPE_CD])
VALUES
  ('PAK-L',
  '2025-01-01 00:00:00.000',
  NULL,
  'Package Loan',
  'OFFER');
-- Check the insert is executed without any error.
IF @@ERROR <> 0
  GOTO FailedTran;

COMMIT TRANSACTION

FailedTran:
print N'The product informations are not inserted’;

ROLLBACK TRANSACTION;
```



Transaction Example : Failed or Pass

```
(1 row affected)
Msg 547, Level 16, State 0, Line 16
The INSERT statement conflicted with the FOREIGN KEY constraint "PRODUCT_PRODUCT_TYPE_FK". The conflict occurred in database "Test", table
"dbo.PRODUCT_TYPE", column 'PRODUCT_TYPE_CD'.
The statement has been terminated.
The product informations are not inserted

Completion time: 2025-02-18T16:49:36.2586542+00:00
```

```
SELECT * FROM PRODUCT_TYPE
SELECT * FROM PRODUCT
```

	PRODUCT_TYPE_CD	NAME
1	ACCOUNT	Customer Accounts
2	INSURANCE	Insurance Offerings
3	LOAN	Individual and Business Loans

	PRODUCT_CD	DATE_OFFERED	DATE_RETIRED	NAME	PRODUCT_TYPE_CD
1	AUT	2000-01-01 00:00:00.000	NULL	auto loan	LOAN
2	BUS	2000-01-01 00:00:00.000	NULL	business line of credit	LOAN
3	CD	2000-01-01 00:00:00.000	NULL	certificate of deposit	ACCOUNT
4	CHK	2000-01-01 00:00:00.000	NULL	checking account	ACCOUNT
5	MM	2000-01-01 00:00:00.000	NULL	money market account	ACCOUNT
6	MRT	2000-01-01 00:00:00.000	NULL	home mortgage	LOAN
7	SAV	2000-01-01 00:00:00.000	NULL	savings account	ACCOUNT
8	SBL	2000-01-01 00:00:00.000	NULL	small business loan	LOAN

UNIVERSITY OF DERBY

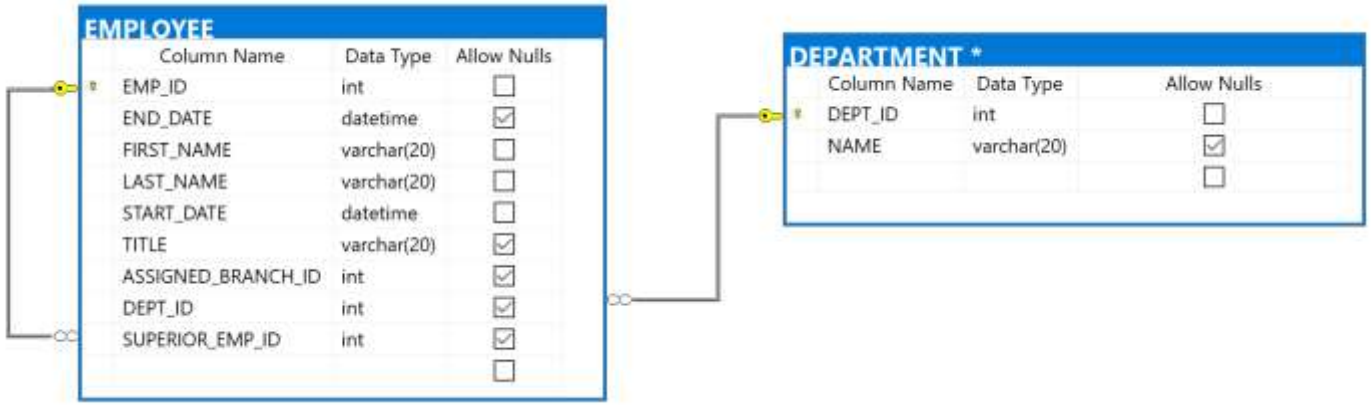
Common Table Expression (CTE)

- **CTE** is powerful mechanism to store results of **SELECT**, **INSERT**, **UPDATE**, **DELETE**, or **MERGE**
- The results are **named** and are accessible within the **local scope**.
- The results are **temporary** and lost after loosing the scope
- Key Advantages
  - **Simplifies Complex Queries** – Helps break down intricate queries into a more manageable structure, improving readability.
  - **Enhances Code Reusability** – Can be referenced multiple times within the main query, avoiding redundant subqueries.
  - **Better Organization for Reporting** – Useful for complex reports that pull data from multiple tables or schema parts.
- Syntax

```
WITH expression_name[(column_name [,...])]
AS
    (CTE_definition)
SQL_statement;
```

Common Table Expression (CTE) – Example (1/2)

- Consider the schema (from Test database)

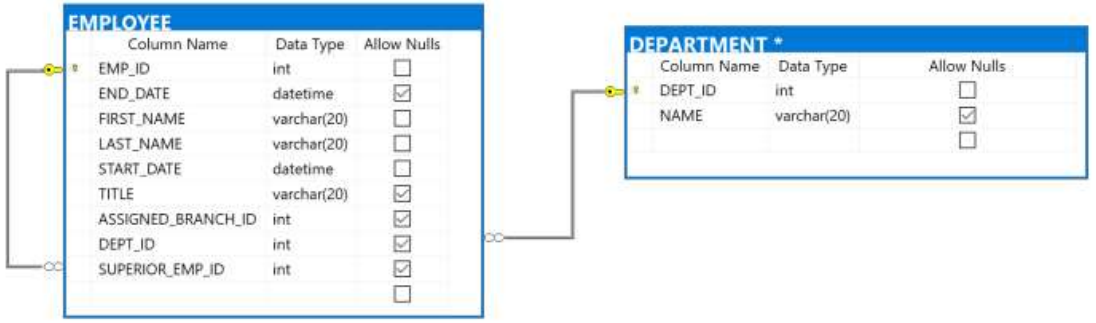


- Creating CTE with name ‘cte_DeptEmployees’ that holds Department name and total employees in each department.

	Department	Total Employees
1	Administration	3
2	Loans	1
3	Operations	14

Common Table Expression (CTE) – Example (2/2)

- Script Code
  - `cte_DeptEmployees`




```
WITH cte_DeptEmployees (Department,
[Total Employees]) AS
(
 SELECT D.NAME, COUNT(D.NAME)
 FROM EMPLOYEE E
 INNER JOIN DEPARTMENT D
 ON E.DEPT_ID = D.DEPT_ID
 GROUP BY D.NAME
)

SELECT * FROM cte_DeptEmployees;
```

	Department	Total Employees
1	Administration	3
2	Loans	1
3	Operations	14

UNIVERSITY OF DERBY

Cursors

- Enable processing result sets **ONE** row **AT** a **TIME**.
- Useful mechanism when accessing and processing required **row-by-row**.
- A ***cursor*** data type is used – for declaring as variable or using as a parameter in functions or stored procedures

- **Supported Operations**:

✓ DECLARE CURSOR **|** OPEN **|** FETCH **|** CLOSE **|** DEALLOCATE

✓ UPDATE **|** DELETE **|** SET **|** CREATE PROCEDURE **|** DECLARE @variable

https://learn.microsoft.com/en-us/sql/t-sql/language-elements/cursors-transact-sql?view=sql-server-ver16

UNIVERSITY OF DERBY

Cursor Support: Built-in Functions and Stored Procedures

Function/Procedure	Description (Brief)
@@CURSOR_ROWS	Returns the number of rows in the current cursor.
CURSOR_STATUS	Checks the status of a cursor.
@@FETCH_STATUS	Returns the status of the last fetch operation.
sp_cursor_list	Lists all open cursors in the session.
sp_describe_cursor	Provides details about a specific cursor.
sp_describe_cursor_columns	Returns column metadata of a cursor.
sp_describe_cursor_tables	Lists tables referenced by a cursor.

https://learn.microsoft.com/en-us/sql/t-sql/language-elements/cursors-transact-sql?view=sql-server-ver16

UNIVERSITY OF DERBY

Declare Cursor

- **T-SQL Extended Syntax:**

```
DECLARE cursor_name CURSOR [ LOCAL | GLOBAL ]
    [ FORWARD_ONLY | SCROLL ]
    [ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
    [ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
    [ TYPE_WARNING ]
    FOR select_statement
    [ FOR UPDATE [ OF column_name [ , ...n ] ] ]
[ ; ]
```


https://learn.microsoft.com/en-us/sql/t-sql/language-elements/declare-cursor-transact-sql?view=sql-server-ver16 [Last accessed: 01-Mar-2025]

Cursor Declaration/Use – Example (Basic Forward Only)

DEPARTMENT

Column Name	Data Type	Allow Nulls
DEPT_ID	int	☐
NAME	varchar(20)	☐
		☐

	DEPT_ID	NAME
1	1	Operations
2	2	Loans
3	3	Administration
4	4	IT
5	5	Finance
6	6	Research

- **Cursor for Department**

```
-- Cursor Declaration
DECLARE dept_cursor CURSOR
  FOR SELECT * FROM DEPARTMENT;
```

- **Cursor open (It's must before use)**

```
-- Before processing open the cursor
OPEN dept_cursor;
```

- **Fetch/read rows (One row at time – here 1st)**

```
-- Fetching rows (1st row - for 1st time execute)
FETCH NEXT FROM dept_cursor;
```

	DEPT_ID	NAME
1	1	Operations

- **Cursor close & deallocate** (Must close if re-use from start – free memory explicitly)

```
-- Close the cursor and Deallocate the memory explicitly
CLOSE dept_cursor;
DEALLOCATE dept_cursor;
```

Cursor Declaration/Use – Example (Basic Forward Only) – Complete code with Two rows

```
-- Cursor Declaration
DECLARE dept_cursor CURSOR
FOR SELECT * FROM DEPARTMENT;

-- Before processing open the cursor
OPEN dept_cursor;

-- Fetching rows (1st row - for 1st time execute)
FETCH NEXT FROM dept_cursor;

-- Fetching rows ( one at a time ) - second row from result
FETCH NEXT FROM dept_cursor;

-- Close the cursor and Deallocate the memory explicitly
CLOSE dept_cursor;
DEALLOCATE dept_cursor;
```

	DEPT_ID	NAME
1	1	Operations

	DEPT_ID	NAME
1	2	Loans

Cursor Example – Use While and @@FETCH_STATUS

```
-- Cursor Declaration
DECLARE dept_cursor CURSOR
  FOR SELECT * FROM DEPARTMENT;

-- Before processing open the cursor
OPEN dept_cursor;

-- Fetch 1st row
FETCH NEXT FROM dept_cursor;

-- Fetching other rows - using While loop
WHILE @@FETCH_STATUS = 0
BEGIN
  FETCH NEXT FROM dept_cursor;
END

-- Close the cursor and Deallocate the memory explicitly
CLOSE dept_cursor;
DEALLOCATE dept_cursor;
```

	DEPT_ID	NAME
1	1	Operations

	DEPT_ID	NAME
1	2	Loans

	DEPT_ID	NAME
1	3	Administration

	DEPT_ID	NAME
1	4	IT

	DEPT_ID	NAME
1	5	Finance

	DEPT_ID	NAME
1	6	Research

UNIVERSITY OF DERBY

JSON and T-SQL: Overview

- **What is JSON?**
  - JSON (JavaScript Object Notation) is a lightweight, semi-structured data format used for data representation.
- **Key Aspects:**
  - Easy to read and write for both humans and machines.
  - Widely used for data exchange between applications.
- **JSON in SQL Server:**
  - Supported since SQL Server 2016.
  - Provides built-in functions to read, parse, and modify JSON data.

UNIVERSITY OF DERBY

JSON Example: Details about Department

- Budget, multiple contacts, and location details.

```
{
    "dept_budget": 500000,
    "dept_contacts": [
        "+44-555-1234",
        "+44-555-5678"
    ],
    "dept_location": {
        "building": "Main Office",
        "floor": 3,
        "room": "311"
    }
}
```

UNIVERSITY OF DERBY

JSON functions in T-SQL

Function	Description
ISJSON()	Checks if a string is valid JSON.
JSON_VALUE()	Extracts a scalar value from JSON data.
JSON_QUERY()	Extracts a JSON array or object.
JSON_MODIFY()	Updates JSON data.
OPENJSON()	Converts JSON data into table format.

JSON processing: Creation

```
-- Creating JSON Data
SELECT
    '{
        "dept_budget": 500000,
        "dept_contacts": [
              "+44-555-1234",
              "+44-555-5678"
         ],
         "dept_location": {
              "building": "Main Office",
              "floor": 3,
              "room": "311"
        }
     }' AS JSON_Data;
```

	JSON_Data
1	{ "dept_budget": 500000, "dept_contacts": ["+44-555-1234", "+44-555-5678"], "dept_location": { "building": "Main Office", "floor": 3, "room": "305" } }

UNIVERSITY OF DERBY

JSON processing: Variable, Validity and retrieving

```
-- Storing JSON object to a variable
DECLARE @dept_details NVARCHAR(MAX);
SELECT @dept_details  = '{
                            "dept_budget": 500000,
                            "dept_contacts": [
                                    "+44-555-1234",
                                    "+44-555-5678"
                            ],
                            "dept_location": {
                                    "building": "Main Office",
                                    "floor": 3,
                                    "room": "311"
                            }
                        }'

-- Checking Validity of JSON
SELECT ISJSON(@dept_details) AS IsValidJson;

-- Display the JSON Text
SELECT @dept_details AS JsonObject;
```

	IsValidJson
1	1

	JsonObject
1	{ "dept_budget": 500000, "dept_contacts"...

UNIVERSITY OF DERBY

JSON processing: Scalar value (using Path Expression)

```
-- Storing JSON object to a variable
DECLARE @dept_details NVARCHAR(MAX);
SELECT @dept_details  = '{
                        "dept_budget": 500000,
                        "dept_contacts": [
                                "+44-555-1234",
                                "+44-555-5678"
                        ],
                        "dept_location": {
                                "building": "Main Office",
                                "floor": 3,
                                "room": "311"
                        }
                }'

-- Retrieving scalar value ( here dept_budget)
SELECT
        JSON_VALUE(@dept_details,'$.dept_budget') totalbudget;
```

	totalbudget
1	500000

JSON processing: Scalar Array value (using Path Expression)

```
-- Storing JSON object to a variable
DECLARE @dept_details NVARCHAR(MAX);
SELECT @dept_details  = '{
                        "dept_budget": 500000,
                        "dept_contacts": [
                                "+44-555-1234",
                                "+44-555-5678"
                        ],
                        "dept_location": {
                                "building": "Main Office",
                                "floor": 3,
                                "room": "311"
                        }
                }'

-- Retrieving scalar value from Array ( here first contact)
SELECT
        JSON_VALUE(@dept_details,'$.dept_contacts[0]') FirstContact;;
```

	FirstContact
1	+44-555-1234

UNIVERSITY OF DERBY

JSON processing: Path Expression Operators

Operator	Meaning
$	Represent the root object of the JSON document
.	Access a property of a JSON object.
[n]	Access the nth element of a JSON array.
*	Match all elements in a JSON array
?()	Filter expression to select JSON elements conditionally
@	Reference the current element within a filter expression

JSON processing: Scalar value nested structure

(using Path Expression)

```
-- Storing JSON object to a variable
DECLARE @dept_details NVARCHAR(MAX);
SELECT @dept_details  = '{
                        "dept_budget": 500000,
                        "dept_contacts": [
                                "+44-555-1234",
                                "+44-555-5678"
                        ],
                        "dept_location": {
                                "building": "Main Office",
                                "floor": 3,
                                "room": "311"
                        }
                }'

-- Retrieving scalar value from nested complex object ( here room)
SELECT
        JSON_VALUE(@dept_details,'$.dept_location.room') DeptOfficeRoom;
```

	DeptOfficeRoom
1	311

JSON processing: Complex values (using Path Expression)

```
-- Storing JSON object to a variable
DECLARE @dept_details NVARCHAR(MAX);
SELECT @dept_details  = '{
                                "dept_budget": 500000,
                                "dept_contacts": [
                                        "+44-555-1234",
                                        "+44-555-5678"
                                ],
                                "dept_location": {
                                        "building": "Main Office",
                                        "floor": 3,
                                        "room": "311"
                                }
                        }'

-- Retrieving complex values - array or object (all contacts here)
SELECT
        JSON_QUERY(@dept_details,'$.dept_contacts') Dept_AllContacts;

-- Retrieving complex values - array or object (location details)
SELECT
        JSON_QUERY(@dept_details,'$.dept_location') Dept_LocationDetails;
```

	Dept_AllContacts
1	["+44-555-1234", "+44-555-5678"]

	Dept_LocationDetails
1	{ "building": "Main Office", "floor": 3, "room": "311" }

JSON processing: Display in Table (using Path Expression)

```
-- Storing JSON object to a variable
DECLARE @dept_details NVARCHAR(MAX);
SELECT @dept_details  = '{
                        "dept_budget": 500000,
                        "dept_contacts": [
                                "+44-555-1234",
                                "+44-555-5678"
                        ],
                        "dept_location": {
                                "building": "Main Office",
                                "floor": 3,
                                "room": "311"
                        }
                }'
```

WITHOUT Distinct

	dept_budget	building	buildingFloor	room
1	500000	Main Office	3	311
2	500000	Main Office	3	311
3	500000	Main Office	3	311

WITH Distinct

	dept_budget	building	buildingFloor	room
1	500000	Main Office	3	311

```
-- Retrieving complex values - in table format (dept_budget and location details)
SELECT DISTINCT
        JSON_VALUE(@dept_details, '$.dept_budget') AS dept_budget,
        JSON_VALUE(@dept_details, '$.dept_location.building') AS building,
        JSON_VALUE(@dept_details, '$.dept_location.floor') AS buildingFloor,
        JSON_VALUE(@dept_details, '$.dept_location.room') AS roomFROM
OPENJSON(@dept_details);
```

JSON processing: Display in Table (using Path Expression)

```
-- Storing JSON object to a variable
DECLARE @dept_details NVARCHAR(MAX);
SELECT @dept_details  = '{
                        "dept_budget": 500000,
                        "dept_contacts": [
                                "+44-555-1234",
                                "+44-555-5678"
                        ],
                        "dept_location": {
                                "building": "Main Office",
                                "floor": 3,
                                "room": "311"
                        }
                }'

-- Retrieving complex values including array - in table format (budget, contacts, and location details)
SELECT
        JSON_VALUE(@dept_details, '$.dept_budget') AS dept_budget,
        JSON_VALUE(@dept_details, '$.dept_location.building') AS building,
        JSON_VALUE(@dept_details, '$.dept_location.floor') AS buildingFloor,
        JSON_VALUE(@dept_details, '$.dept_location.room') AS room,
        contacts AS dept_contact
FROM OPENJSON(@dept_details, '$.dept_contacts')
WITH (contacts NVARCHAR(50) '$');
```

	dept_budget	building	buildingFloor	room	dept_contact
1	500000	Main Office	3	311	+44-555-1234
2	500000	Main Office	3	311	+44-555-5678